

MEDIATEK

MT8676 Hypervisor UART User Manual

版本: 1.0
出版日期: 2024-11-20

Use of this document and any information contained therein is subject to the terms and conditions set forth in Exhibit 1. This document is subject to change without notice.

版本记录

版本	日期	作者	描述
1.0	2024-11-20	陈李亮	正式版

目录

版本记录 2

目录 3

图片目录 3

表格目录 4

1 Hypervisor UART 5

 1.1 概述 5

 1.1.1 简单介绍 5

 1.1.2 名词解释 5

 1.2 UART 虚拟化方式 6

 1.2.1 虚拟化说明 6

 1.2.2 UART 虚拟化握手流程 6

 1.2.3 UART 虚拟化特征 7

 1.2.4 测试与调试 8

 1.3 Passthrough 直通方式 8

 1.3.1 直通说明 8

 1.3.2 具体实作 9

 1.3.3 测试和调试 11

 1.3.4 常见问题及故障排除 12

 1.3.5 寻求联发科技帮助 12

附件一 附加条款 13

图片目录

图 1-1. UART 虚拟化布局图 5

图 1-2. 握手流程图 6

图 1-3. SOS 部分改动 9

图 1-4. UOS-DTS 部分改动 10

图 1-5. UOS-KO 部分改动 10

图 1-6. Hypervisor 部分改动 11

表格目录

表 1-1. 名词解释.....5

表 1-2. 代码路径.....6

表 1-3. UART 驱动代码路径.....8

1 Hypervisor UART

1.1 概述

1.1.1 简单介绍

本章节主要介绍 MT8676 Hypervisor UART 的软件以及功能，虚拟化总体布局如图 1-1 所示。

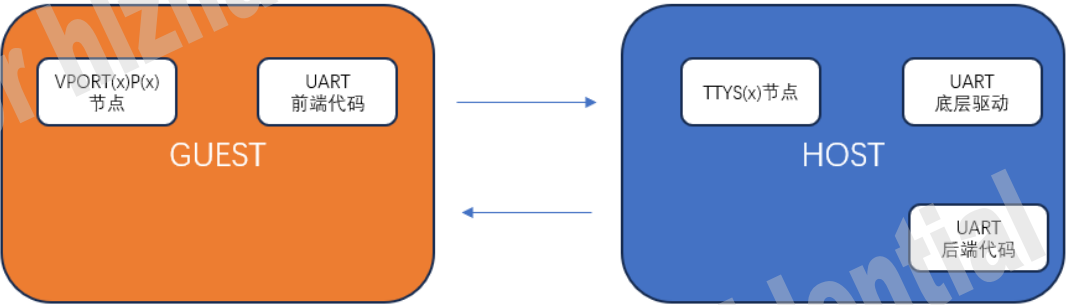


图 1-1. UART 虚拟化布局图

1.1.2 名词解释

表 1-1. 名词解释

缩写	解释
DMA	直接内存访问
GUEST	前端系统
HOST	后端系统
TTYS(x)	直通绑定的 UART 节点
UART	通用异步收发器
VPORT(x)P(x)	前端生成的 UART 虚拟节点

1.2 UART 虚拟化方式

1.2.1 虚拟化说明

对于多 OS 系统来说，由于多个外部 devices 需要用到 UART 进行通信，但受平台 UART 数量限制的影响无法实施，因此考虑对 UART 模块进行虚拟化使得不同 devices 可以在不同 OS 系统中使用 UART 模块，具体代码路径参考表 1-1。

表 1-2. 代码路径

前端路径	后端路径
kernel/drivers/char/virtio_console.c	kernel/modules/mt8676/virt/grt/vhost_uart/uart.c

1.2.2 UART 虚拟化握手流程

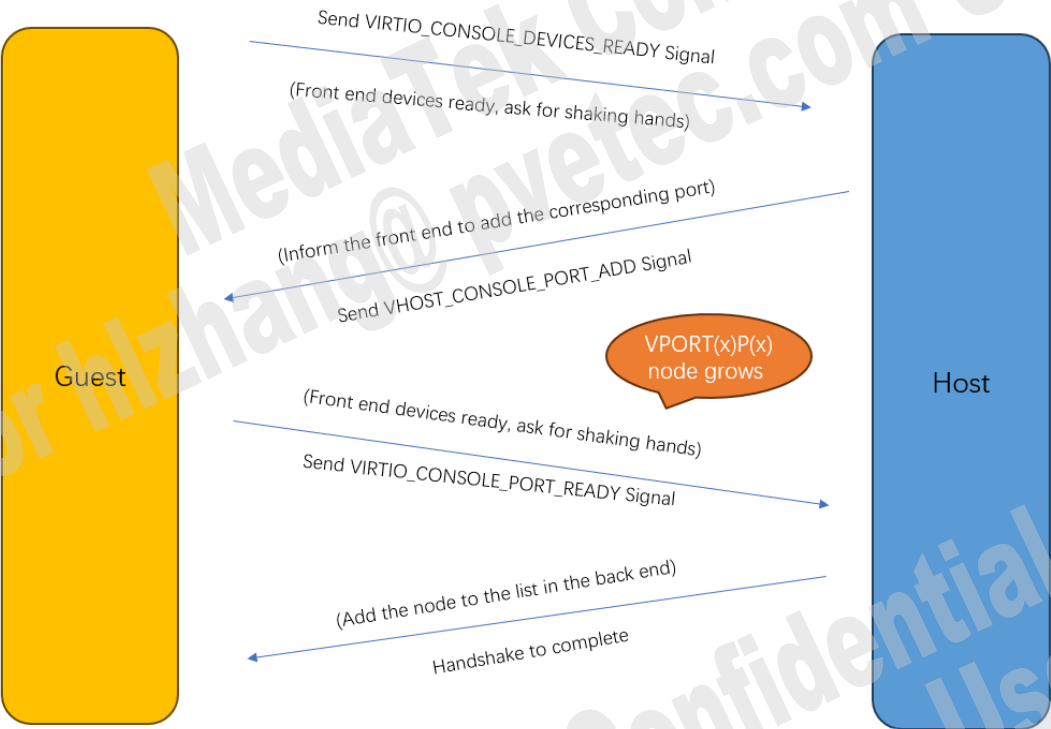


图 1-2. 握手流程图

对于多 OS 系统虚拟化而言，最关键的是建立前后端的通信连接。如图 1-2 所示，总共可分为四部分：

1. 前端 (Guest) 发送 VIRTIO_CONSOLE_DEVICES_READY 事件信号给后端 (Host) , 代表着前端 (Guest) devices 已准备好。
2. 后端 (Host) 收到信号后会往前端 (Guest) 发送 VHOST_CONSOLE_PORT_ADD 事件信号给前端 (Guest) , 代表后端 (Host) 收到请求连接的信号需要前端添加相应的 PORT 节点。
3. 当前端 (Guest) 收到信号后会往后端 (Host) 发送 VIRTIO_CONSOLE_PORT_READY 事件信号, 并且此刻会在前端 (Guest) 生成虚拟化节点 VPORT(x)P(x), x 是随机生成的号码。
4. 当后端 (Host) 收到信号后会往后端 (Host) 将生成的节点添加至列表中, 此刻代表整体握手流程结束。

1.2.3 UART 虚拟化特征

1. 前端事件信号:

- a) VIRTIO_CONSOLE_DEVICE_READY
设备准备
- b) VIRTIO_CONSOLE_PORT_ADD
节点添加
- c) VIRTIO_CONSOLE_PORT_REMOVE
移除节点
- d) VIRTIO_CONSOLE_PORT_READY
节点准备
- e) VIRTIO_CONSOLE_PORT_OPEN
开启节点

2. 后端事件信号:

- a) VHOST_CONSOLE_DEVICE_READY
设备准备
- b) VHOST_CONSOLE_PORT_ADD
节点添加
- c) VHOST_CONSOLE_PORT_REMOVE
移除节点
- d) VHOST_CONSOLE_PORT_READY

节点准备

- e) VHOST_CONSOLE_PORT_OPEN
开启节点
- f) VHOST_CONSOLE_PORT_READ
读取节点

1.2.4 测试与调试

1. 查看前端是否生成了 UART 虚拟节点

```
find . -name vport*
```

2. UART 获取数据，可以使用以下命令接收 UART 数据并通过 console 打印出来

```
cat /dev/vport*
```

3. UART 发送数据，可以使用以下命令让 UART 发送字符串 “123”

```
echo 123 > /dev/vport*
```

1.3 Passthrough 直通方式

1.3.1 直通说明

对于多 OS 而言，每个 OS 下都存在一套 MTK UART 驱动代码，如表 1-3 所示。
UART 的 virt 方式目前支持读/写/开/关四种操作，需要通过 Passthrough 直通的方式来进行修改波特率/开启软控流/开启硬控流的操作，也就是在对应的 OS 下开启 UART 驱动代码。

表 1-3. UART 驱动代码路径

系统	UART 路径	APDMA 路径
Android	kernel/kernel_device_modules-6.1/tty/serial/8250/8250_mtk.c	kernel/kernel_device_modules-6.1/dma/mediatek/mtk-uart-apdma.c
Yocto	src/kernel/linux/v6.1_mt8676/co_device_module/drivers/tty/serial/8250/8250_mtk.c	src/kernel/linux/v6.1_mt8676/co_device_module/drivers/dma/mediatek/mtk-uart-apdma.c

1.3.2 具体实作

本次以 L+A+L 系统中的 SOS (Yocto) 的 UART2 Passthrough 到 UOS (Android) 端举例说明。

1.3.2.1 SOS (Yocto)部分改动

SOS (Yocto) 部分改动较小，只需要确保 auto8676p1_64_hyp_sos.dts 文件中的 UART2 节点属性 status 从 “okay” 改为 “disabled” 即可，如图 1-3 所示。

```
diff --git a/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts b/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
index c2df6b5..75ffc9d 100644
--- a/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
+++ b/recipes-kernel/linux/files/auto8676p1_64_hyp_sos.dts
@@ -393,7 +393,7 @@
&uart2 {
    pinctrl-names = "default";
    pinctrl-0 = <&uart2_pins_default>;
-   status = "okay";
+   status = "disabled";
};
```

图 1-3. SOS 部分改动

1.3.2.2 UOS (Android)部分改动

UOS (Android) 改动有两处：

1. dts 修改

UOS (Android) 系统需修改 auto8676p1_64_bsp_vm.dts 文件，将 UART2 节点进行开启，并且将 UART2 对应的 pin 脚切为 UART 模式，使得平台开机后 pin 脚会自动切到 UART 模式上。具体改动如图 1-4 所示。

```

diff --git a/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts b/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
index db4023b90c89..9325e59e8b8f 100644
--- a/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
+++ b/arch/arm64/boot/dts/mediatek/auto8676p1_64_bsp_vm.dts
@@ -1632,6 +1632,27 @@ &backlight_cooler {
    status = "disabled";
};

+&pio {
+    uart2_pins_default: uart2-pins-default {
+        pins-rx {
+            pinmux = <PINMUX_GPIO117_FUNC_URXD2>;
+            input-enable;
+            bias-pull-up;
+        };
+        pins-tx {
+            pinmux = <PINMUX_GPIO121_FUNC_UTXD2>;
+        };
+        pins-rts {
+            pinmux = <PINMUX_GPIO162_FUNC_URTS2>;
+            output-enable;
+        };
+        pins-cts {
+            pinmux = <PINMUX_GPIO161_FUNC_UCTS2>;
+            input-enable;
+        };
+    };
+};
+
&uart0 {
    status = "disabled";
};
@@ -1641,7 +1662,9 @@ &uart1 {
};

&uart2 {
-    status = "disabled";
+    pinctrl-names = "default";
+    pinctrl-0 = <&uart2_pins_default>;
+    status = "okay";
};

```

图 1-4. UOS-DTS 部分改动

2. ko_table 修改

由于在 UOS (Android) 端并未添加 UART 相关 ko 文件，因此还需要在 ko_order_table 文件中添加，如图 1-5 所示。

```

diff --git a/ko_order_table.csv b/ko_order_table.csv
index 7c9b7ee..e0b0dcb 100644
--- a/ko_order_table.csv
+++ b/ko_order_table.csv
@@ -127,6 +127,9 @@ mtk-pmic-keys.ko,./kernel_device_modules-6.1/drivers/input/keyboard/mtk-pmic-k
mtk-kpd.ko,./kernel_device_modules-6.1/drivers/input/keyboard/mtk-kpd.ko,ramdisk,Y,Y,user/userdebug/eng
i2c-mt65xx.ko,./kernel_device_modules-6.1/drivers/i2c/busses/i2c-mt65xx.ko,ramdisk,Y,Y,user/userdebug/eng
spi-mt65xx.ko,./kernel_device_modules-6.1/drivers/spi/spi-mt65xx.ko,ramdisk,Y,Y,user/userdebug/eng
+mtk-uart-apdma.ko,./kernel_device_modules-6.1/drivers/dma/mediatek/mtk-uart-apdma.ko,ramdisk,Y,Y,user/userdebug/eng
+uarthub_drv.ko,./kernel_device_modules-6.1/drivers/misc/mediatek/uarthub/uarthub_drv.ko,vendor,Y,Y,user/userdebug/eng
+8250_mtk.ko,./kernel_device_modules-6.1/drivers/tty/serial/8250/8250_mtk.ko,ramdisk,Y,Y,user/userdebug/eng
extdev_io_class.ko,./kernel_device_modules-6.1/drivers/misc/mediatek/subpmic/extdev_io_class.ko,ramdisk,Y,Y,user/userdebug/eng
subpmic-dbg.ko,./kernel_device_modules-6.1/drivers/misc/mediatek/subpmic/subpmic-dbg.ko,ramdisk,Y,Y,user/userdebug/eng
rt5133-regulator.ko,./kernel_device_modules-6.1/drivers/regulator/rt5133-regulator.ko,ramdisk,Y,Y,user/userdebug/eng

```

图 1-5. UOS-KO 部分改动

1.3.2.3 Hypervisor 部分改动

由于 Hypervisor 部分将中断号全部开启在 SOS (Yocto) 端，需要把 UOS (Android) 端所使用到的中断号开启，因此需要在 uos_alps_pv8676.lua 文件中将 UART2 的中断号开在 UOS 端，如图 1-6 所示。

```
if Android_USB_SWITCH_value == "1" then
    print("use android usb")
    interrupt = {48, 70, 192, 193, 231, 255, 256, 284, 634, 635, 645, 646, 921, 923, 924, 929}
    for i = 1, #interrupt do
        uos_config:setVgicIrq(interrupt[i])
    end
end
```

图 1-6. Hypervisor 部分改动

1. UART2 的中断号在 MT6897.dts 文件中可以查询为 252，而写到 LUA 文件中需要增加 32，因此 UART2 的中断号在 uos_alps_pv8676.lua 文件中为 284。
2. 当 UART2 开启 DMA 模式时，DMA 拥有 DMA TX/DMA RX 两个通道，因此 UART2 的 DMA 查表可以查询为 223, 224，通过增加 32 可得到在 lua 文件中为 255, 256。

1.3.3 测试和调试

在对应开启 UART 驱动的 OS 下执行如下操作进行测试：

1. 设置波特率 921600，8 位数据位，1 位停止位，无奇偶校验，如 UART2

```
stty -F /dev/ttyS2 ispeed 921600 ospeed 921600 cs8
```

2. 获取当前 UART 波特率以及数据位配置信息

```
stty -F /dev/ttyS2 -a
```

3. UART 获取数据，可以使用以下命令接收 UART2 数据并通过 console 打印出来

```
cat /dev/ttyS2
```

4. UART 发送数据，可以使用以下命令让 UART2 发送字符串"123"

```
echo 123 > /dev/ttyS2
```

1.3.4 常见问题及故障排除

1.3.4.1 改完后 Android 起不来

1. 请检查平台中（非 codebase）uos_alps_pv8676.lua 文件是否包含了 UART 中断号，具体可在 shell 中敲击命令：

```
vi /vendor/etc/hyper_android/uos_alps_xx.lua
```

请务必保证平台中的 lua 文件包含 UART 所需的中断号。

2. 如开机 log 中出现类似于如下 log

```
init: [name:main&]8250_mtk: Unknown symbol UARTHUB_md_adsp_fifo_ctrl (err -2)
```

请检查 ko_order_table 文件是否正确修改到，并且 3 个 ko 是否有如图 1-5 方式正确添加。

1.3.4.2 改完后测试 UART 未成功

1. 请检查修改的 UOS/SOS 对应的 dts 是否正确，前文是以 L+A+L 系统为例，如是 L+A 系统则 dts 文件不一样，请确认是否正确修改 dts 文件。
2. 请检查 dts 配置是否正确，请使用 Linux 命令检查 UART 引脚 mode 是否切换至 UART 模式

Kernel 里查看 PIN 的状态，可输入以下命令进行查询：

```
# cat /proc/mtk_gpio/soc.pinctrl
# cd /proc/mtk_gpio/
# cat soc.pinctrl
```

e.g.,

```
# cat /proc/mtk_gpio/soc.pinctrl
PIN: [MODE] [DIR] [DOUT] [DIN] [PULL_EN] [PULL_SEL] [IES] [SMT] [DRIVE] ( [R1] [R0] )
0: 0 0 0 0 1 0 1 0 0
1: 0 0 0 0 1 0 1 0 0
2: 0 1 1 1 1 0 1 0 0
3: 6 0 0 0 1 0 1 0 0
```

1.3.5 寻求联发科技帮助

若经过前文的排查仍无法解决您的问题，向联发科技寻求帮助时请提供相关日志/DTS/cat mt_gpio 等信息。

附件一 附加条款

您(或您所代表的公司或其他法律主体, 以下合称「您」)是否得取得、下载与使用本文件及其相关信息皆应以您接受本附加条款为先决要件。您对于本文件之使用、取得或下载即表示您已接受本附加条款并同意受本附加条款之拘束。若您不同意受本附加条款之拘束, 您将不得使用、取得或下载本文件并应立即删除或毁弃所有本文件之副本。

本文件含有联发科技股份有限公司及其关联公司(以下合称「联发科技」)或其授权人之机密信息及专有信息, 仅供您为本文件所描述之联发科技芯片组作内部使用而不得用于其他任何目的(包括但不限于确认或提供支持任何对联发科技、其供货商及/或其直接或间接客户所提潜在专利侵权主张的证据)。禁止未经授权使用或揭露本文件及其中所含信息。您同意赔偿联发科技因您未经授权使用或揭露本文件及其中所含信息之一部或全部导致联发科技所受之任何损失或损害。

联发科技及其授权人保有本文件的所有权及组成所有权之各项权利且未针对任何知识产权授权(无论明示或默示、透过禁反言原则或其他方式)。联发科技得随时变更本文件内容而无须另行通知。联发科技无须承担与使用或信赖本文件相关或因使用或信赖本文件致生之任何责任, 包括但不限于间接损害或附带损害赔偿責任。

本文件及联发科技所提供关于本文件之任何其他材料、信息或技术支持, 均以现况交付为准, 联发科技不负任何明示、默示、法定或其他形式之担保责任, 特别是适销性、不侵权、针对特定用途之适用性、完整性或正确性之担保责任或任何由贸易惯例或交易、履行过程所生之担保责任。对于联发科技为符合您所提规格或遵循特定标准或标准组织的要求所作之交付物, 联发科技亦不应承担任何责任。

于未对前述条款造成限制之情形下, 联发科技不对其产品任何特定用途之适用性承担任何保证或担保责任, 亦不承担任何因产品、电路或软件之应用或使用致生之赔偿责任。您同意您应单独承担关于整合您的产品与联发科技产品所涉及之设计、验证与测试之所有责任, 并确保前述整合产品符合相关标准及任何安全性要求或其他要件。

本附加条款及所有与本附加条款或本文件相关之行为应以台湾法律管辖、解释与阐明, 不适用冲突法原则。